

SYSTEM
OVERVIEW

- PDF Knowledge Base
- LangGraph Workflow
- ChromaDB Vector Store
- HITL Escalation
- Streamlit Web UI

HIGH-LEVEL DESIGN DOCUMENT

RAG Support Assistant

Retrieval-Augmented Generation
+ LangGraph + Human-in-the-Loop

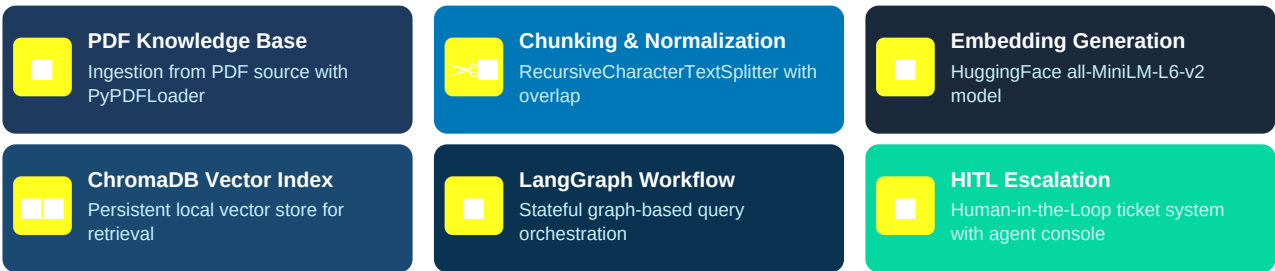
Document Type:	High-Level Architecture Design
Version:	1.0
Classification:	Technical Reference

1 System Overview

Problem Definition

This project implements a Retrieval-Augmented Generation (RAG) based customer support assistant that answers questions from a PDF knowledge base. A general-purpose LLM should not answer support queries from memory because that leads to hallucinations, inconsistent policy answers, and poor traceability. This system constrains answers using retrieved document chunks and routes uncertain cases to a human support path.

Scope of the System



Out of Scope

- Multi-document corpus management
- External helpdesk or CRM integration
- User authentication and session persistence
- Feedback learning loop
- Production deployment infrastructure

2 Architecture Diagram

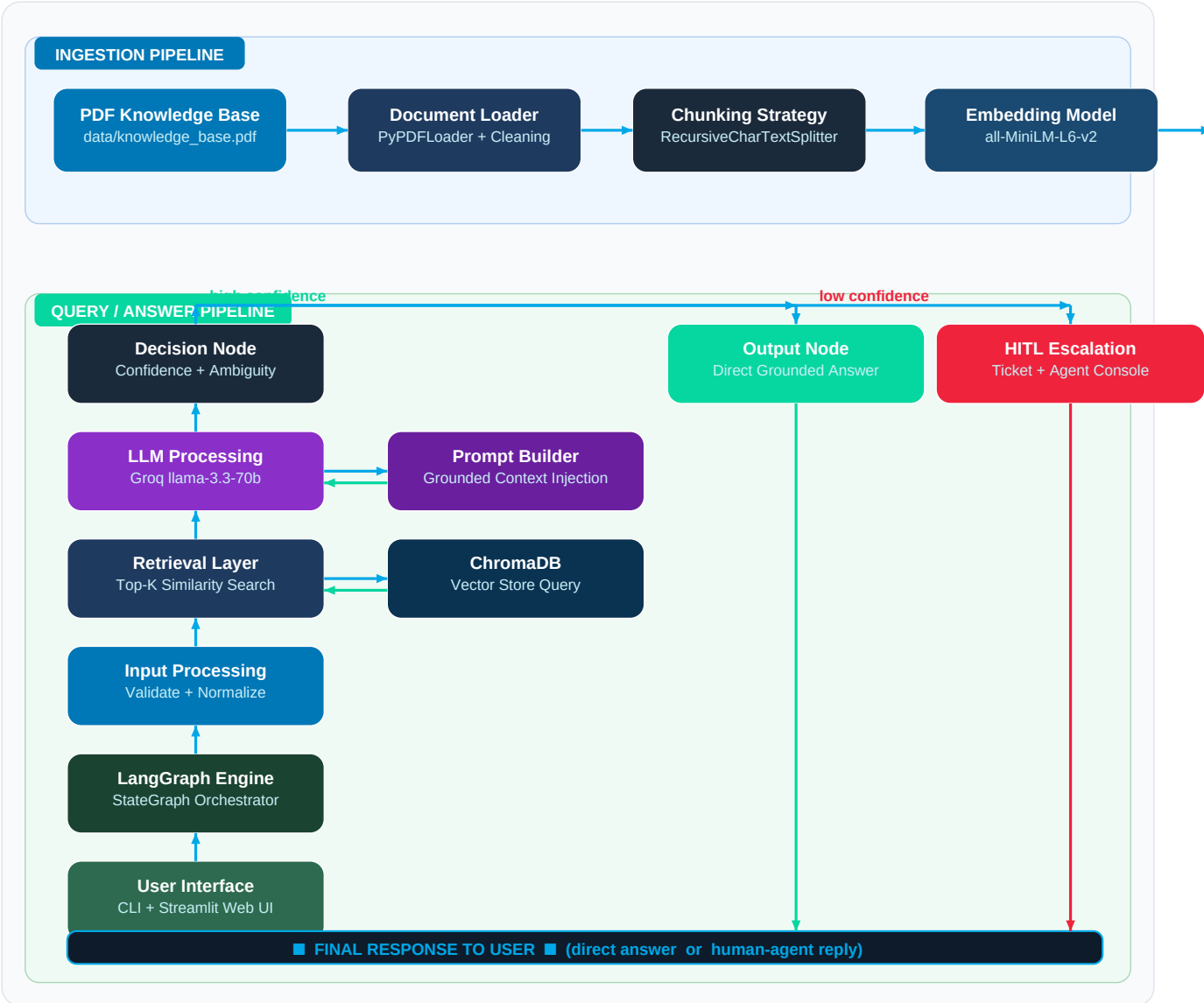


Figure 1 — End-to-end system architecture showing the Ingestion Pipeline (top) and the Query/Answer Pipeline (bottom) with conditional HITL routing.

3 Component Description

Document Loader

File: ingestion.py

- Implemented with PyPDFLoader.
- Loads PDF page by page into LangChain Document objects.
- Preserves source metadata: page number and source path.

Chunking Strategy

File: ingestion.py

- RecursiveCharacterTextSplitter with chunk_size=1000, overlap=200.
- Separators prioritize paragraph and sentence boundaries.
- Each chunk tagged with chunk_index metadata.

Embedding Model

File: vector_store.py

- sentence-transformers/all-MiniLM-L6-v2 via HuggingFaceEmbeddings.
- Embeddings normalized for cosine similarity alignment.
- Lightweight and fast for local ingestion.

Vector Store

File: vector_store.py

- ChromaDB with local persistence under ./chroma_db.
- Collection configured with hnsw:space = cosine.
- Stores chunk embeddings and associated metadata.

Retriever

File: retriever.py

- similarity_search_with_relevance_scores for top-k chunks.
- Confidence = maximum retrieved similarity score.
- Ambiguity heuristics applied to detect vague queries.

LLM

File: llm_service.py

- Groq-hosted ChatGroq: llama-3.3-70b-versatile.
- Prompt grounded — forbids unsupported claims.
- Good latency-performance tradeoff for interactive QA.

LangGraph Workflow

File: workflow.py

- Nodes: input_processing, retrieve_and_answer, decision, output, escalation.
- Explicit routing between direct answer and HITL escalation paths.
- State-centric design makes workflow inspectable and extensible.

HITL Module

File: hitl_store.py + streamlit_app.py

- LangGraph decides when escalation is required.
- Streamlit creates persistent local ticket on escalation.
- Agent resolves ticket; answer delivered into original session.

4 Data Flow

Ingestion Flow: PDF → Vector Store

#	Stage	Detail
1	Run ingestion	python ingest.py run
2	PDF load	Page-by-page via PyPDFLoader into Document objects
3	Normalize	Collapse noisy whitespace in each page text
4	Chunk	Split into overlapping chunks with metadata
5	Embed	Each chunk vectorized with all-MiniLM-L6-v2
6	Store	Embeddings + metadata persisted to ChromaDB

Query Lifecycle: User Question → Final Answer

1	User submits question through CLI or Streamlit.
2	LangGraph initializes workflow state with raw query.
3	input_processing_node cleans and validates the query.
4	retrieve_and_answer_node retrieves top-k chunks from Chroma.
5	Retrieved chunks injected into grounded LLM prompt.
6	LLM produces a grounded answer or indicates insufficient context.
7	decision_node evaluates confidence and ambiguity flags.
8	Route to output_node (direct) or escalation_node (HITL).
9	If escalated via Streamlit, ticket created with full context.
10	Human agent resolves ticket from Agent View.
11	Final answer delivered from LLM or human agent reply.

5

Technology Choices

ChromaDB

- Lightweight and easy to run locally for assignment-scale systems.
- Native LangChain integration.
- Persistent local storage without separate DB ops overhead.
- Suitable for single-PDF and small-to-medium corpus.

LangGraph

- Explicit workflow representation for query processing.
- Clean separation of retrieval, answer generation, decisioning, escalation.
- Easier to extend than a single monolithic chain.

Groq + LLaMA 3.3 70B

- Good latency-performance tradeoff for interactive QA.
- Simple LangChain integration through ChatGroq.
- Avoids running local inference infrastructure.

all-MiniLM-L6-v2

- Compact sentence-transformer model.
- Fast enough for local ingestion.
- Normalised embeddings align with cosine similarity in Chroma.

6 Scalability Considerations

Concern	Current Behaviour	Scaling Path
Large Documents	Single PDF path; chunk count grows linearly with document size.	Batch ingestion, metadata filtering, multiple collections, incremental updates.
Query Load	CLI rebuilds vector store and workflow per request.	Reuse graph/retriever, run behind API service, concurrent request handling.
Latency	Embedding compute + vector retrieval + external LLM API call.	Cache initialised services, tune top_k, optionally replace embedding model.